

矩阵

矩阵就是一个数字列阵，一个n行r列的矩阵可以表示为：

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{pmatrix}$$

矩阵的简单运算

矩阵加法：

矩阵加法很简单，就是把两个矩阵每个相同位置上的相加，第一个矩阵为A，第二个矩阵为B，那么就是 $A_{ij}+B_{ij}$ 。

很明显，两个矩阵做加减法必须要满足的条件是这两个矩阵的行列相同，否则就不可以进行加减运算，最终的得到的矩阵的行列和原矩阵相同。

$$\begin{bmatrix} 1 & 3 \\ 1 & 0 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 7 & 5 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 1+0 & 3+0 \\ 1+7 & 0+5 \\ 1+2 & 2+1 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 8 & 5 \\ 3 & 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 3 \\ 1 & 0 \\ 1 & 2 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 7 & 5 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 1-0 & 3-0 \\ 1-7 & 0-5 \\ 1-2 & 2-1 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ -6 & -5 \\ -1 & 1 \end{bmatrix}$$

矩阵的简单运算

矩阵乘法：

矩阵乘法稍微复杂一点，比如 $A \times B$ ，相当于A矩阵的第i行的每个数与B矩阵的第j列对应位置的每个数相乘当做新矩阵[i, j]位置的值。

$$A \times B = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \times B = \begin{pmatrix} a_1 \times B \\ a_2 \times B \\ a_3 \times B \end{pmatrix} = \begin{pmatrix} a_{11} \times b_1 + a_{12} \times b_2 + a_{13} \times b_3 \\ a_{21} \times b_1 + a_{22} \times b_2 + a_{23} \times b_3 \\ a_{31} \times b_1 + a_{32} \times b_2 + a_{33} \times b_3 \end{pmatrix}$$

$$\begin{bmatrix} 2 & 4 & 5 \\ 1 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 3 & 4 \\ 1 & 3 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 2 \times 3 + 1 \times 4 + 2 \times 5 & 2 \times 4 + 3 \times 4 + 5 \times 1 \\ 1 \times 3 + 3 \times 1 + 4 \times 2 & 1 \times 4 + 3 \times 3 + 4 \times 1 \end{bmatrix} = \begin{bmatrix} 20 & 25 \\ 14 & 17 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 4 \\ 1 & 3 \\ 2 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 4 & 5 \\ 1 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 \times 2 + 1 \times 4 & 3 \times 4 + 1 \times 3 & 3 \times 5 + 1 \times 4 \\ 1 \times 2 + 3 \times 1 & 1 \times 4 + 3 \times 3 & 1 \times 5 + 3 \times 4 \\ 2 \times 2 + 1 \times 1 & 2 \times 4 + 1 \times 3 & 2 \times 5 + 1 \times 4 \end{bmatrix} = \begin{bmatrix} 10 & 15 & 19 \\ 5 & 13 & 17 \\ 5 & 11 & 14 \end{bmatrix}$$

矩阵的简单运算

所以矩阵的乘法首先必须要满足的条件是：

如果A是一个n行r列的矩阵，B是一个r行m列的矩阵，用A矩阵每一行去乘以B矩阵的每一列，那么最终会得到一个n行m列的矩阵。

矩阵乘法不满足交换律，首先是r行m列的矩阵无法和一个n行r列的矩阵相乘，因为不满足条件；其次，一个n行m列的矩阵*一个m行n列的矩阵得到一个n行n列的矩阵；一个m行n列的矩阵*n行m列的矩阵得到一个m行m列的矩阵，完全没有任何联系。

但是矩阵的乘法满足结合律，即 $(A*B)*C=A*(B*C)$

单位矩阵

那么思考一下，一个方阵A(n行n列)要乘以一个什么矩阵才能等于他本身？

$$\begin{bmatrix} 2 & 3 \\ 1 & 5 \end{bmatrix} \times \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 1 & 5 \end{bmatrix}$$

① $2a+3c=2$
② $3b+3d=3$
③ $a+5c=1$
④ $b+5d=5$

①③联立: $\begin{cases} a=1 \\ c=0 \end{cases}$
②④联立: $\begin{cases} b=0 \\ d=1 \end{cases}$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

实际上对于任意的一个矩阵，他乘以只有主对角线为1的矩阵，都等于它本身。只有主对角线为1，其他都为0的矩阵，称之为单位矩阵。

方阵乘幂

方阵乘幂是指， A 是一个方阵，将 A 连乘 N 次，即 $C=A^n$ 。

注意：只有 A 是方阵的时候才可以乘幂。

那么我们应该如何快速进行乘幂计算呢？

因为矩阵乘法满足结合律，那么我们完全可以考虑快速幂的做法来进行方阵快速乘幂。

```
node js(node t1,node t2)
{
    node t3;
    for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++)
    t3.map[i][j]=0; //一定要初始化
    for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++)
    for(int k=1;k<=n;k++)
    t3.map[i][j]=(t3.map[i][j]+t1.map[i][k]*t2.map[k][j])%p;
    return t3;
}
```

方阵乘幂

只要矩阵乘法会了，矩阵快速幂就简单了。参照整数的快速幂。

```
node quick(int x)
{
    if(x==0)
        return e; // 返回一个单位矩阵
    node ans=quick(x/2);
    ans=js(ans,ans); // 两个矩阵相乘
    if(x&1)
        ans=js(ans,a);
    return ans;
}
```

斐波那契数列

看上去矩阵快速幂有点鸡肋，在生活中并没有太大的实际意思，但是如果我们能把某种状态转移成一个矩阵，再把他的递推式转移成矩阵的乘幂，那么我们就可以log的时间复杂度计算出一个递推式的答案。

比如：求解斐波那契数列的第n项， $n \leq \text{long long}$ 。

很明显，线性是无法求解的，那么是不是就没有更快的算法了呢？并不是。我们可以尝试是否可以把当前状态转移成一个矩阵，把递推公式转移成矩阵乘幂。转移成状态很简单，因为我们只有知道前两个状态才能知道后一个状态，那么矩阵可能就是

$$\begin{bmatrix} F[i] \\ F[i-1] \end{bmatrix}$$

$$\begin{bmatrix} F[i+1] \\ F[i] \end{bmatrix}$$

斐波那契数列

左边是一个2行1列的矩阵，右边也是一个2行1列的矩阵，如果要成一个数字矩阵(因为涉及到幂，所以只能是方阵)，有两种可能，第一种是1行1列，第二种是2行2列(在前面和在后面)，1行1列肯定没有办法满足条件。

$$\begin{bmatrix} F[i+1] \\ F[i] \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} * \begin{bmatrix} F[i] \\ F[i-1] \end{bmatrix}$$

$$a * F[i] + b * F[i-1] = F[i+1]$$

$$c * F[i] + d * F[i-1] = F[i], \text{ 那么结果就出来了。}$$

$$a=1, b=1, c=1, d=0;$$

$$\begin{bmatrix} F[i] \\ F[i-1] \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-2} * \begin{bmatrix} F[2] \\ F[1] \end{bmatrix}$$



斐波那契数列

求斐波那契数列前 n 项之和, $n \leq 2 \times 10^9$, 答案对 m 取余,
 $m \leq 10^9 + 10$;

输入: 5 1000

输出: 12



斐波那契数列

该题先一看，太简单了，直接用矩阵加速斐波那契数列，但是又一看，求和...对每一项进行矩阵快速幂求解？还不如暴力递推呢。所以我们要换一种思路。

比如找下斐波那契数列 $S[i]$ 和 $S[i-1]$ 的递推关系。

构造的矩阵里面肯定要有 $S[i]$, $F[i]$ ，但是只有这两个是肯定不够的，因为没有办法转换，那么肯定需要一个类似 $F[i-1]$ 这样的东西，才能把他转换成下一项。

$$\begin{bmatrix} S[i] \\ F[i] \\ F[i-1] \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} * \begin{bmatrix} S[i-1] \\ F[i-1] \\ F[i-2] \end{bmatrix}$$

斐波那契数列

很容易计算出第一行分别是(1, 1, 1), 第二行是(0, 1, 1), 第三行是(0, 1, 0), 我们在计算过程发现, 当第一行的值确定之后, 对后面是没有任何影响的, 相当于每行之间是相互独立的, 所以以后遇到类似问题就比较容易推导了。

当然, 很多时候推导的公式不止一种, 如果你的矩阵不是($S[i]$, $F[i]$, $f[i-1]$), 那么你的数字矩阵也会有一点变化。



佳佳的 Fibonacci

求 $T[i] = (F[1] + 2 * F[2] + \cdots + n * F[n]) \% m$ 的答案



佳佳的 Fibonacci

和上题一样，首先我们已知 $T[i]$ 和 $F[i]$ 的关系，那么就可以推导出 $T[i]$ 和 $T[i-1]$ 的关系， $T[i]=T[i-1]+iF[i]$ 但是很复杂， $iF[i]$ 中两个变量相乘，那我们看下是否可以推导出 $T[i]$ 和 $S[i]$ 的递推关系。

$$T[n]=F[1]+2F[2]+3F[3]+\cdots nF[n];$$

$$nS[n]=nF[1]+nF[2]+\cdots nF[n];$$

$$nS[n]-T[n]=(n-1)F[1]+(n-2)F[2]+\cdots F[n];$$

$$\text{我们设 } P[n]=nS[n]-T[n];$$

那么 $P[n]=P[n-1]+S[n-1]$ ，那么这个公式就和上一道题是一样的了。接下来就可以来推导矩阵了。

佳佳的 Fibonacci

矩阵很容易推导出来

1 1 0 0 0 1 1 1 0 0 1 1 0 0 1 0

以后遇到求解一个多项式的结果，如果他是一个递推式，那么一定可以转换成矩阵的形式。即：

$$F[n] = a_1 * F[n-1] + a_2 * F[n-2] + \dots + a_k * F[n-k].$$

这是一个k阶系数线性递推关系。

并且设计到的常数方阵也是k阶的。

但是，如果给的不是一个递推式呢？一般情况下，如果一个等式他是发散序列，那么他很难写成递推式的形式，如果他是线性或者收敛行的序列，那么他就可以构造递推式。对于发散序列，可以先把他转换成一个线性或者收敛的序列。



练习题

洛谷 1962 斐波那契数列

L0J 10221 斐波那契前n项和

L0J 10222 佳佳的Fibonacci

洛谷 2233 公交车路线

洛谷 2886 牛继电器

BZOJ 1009 GT考试

BZOJ 1297 迷路

BZOJ 2973 石头游戏



高斯消元

高斯消元是一种求解线性方程组的方法。所谓线性方程组，是由M个N元一次方程共同构成的。线性方程组的所有系数可以写成一个M行N列的“系数矩阵”，再加上每个方程等号右侧的常数，可以写成一个M行N+1列的“增广矩阵”。

$$\begin{cases} x + 2y + z = 7 & A \\ 2x - y + 3z = 7 & B \\ 3x + y + 2z = 18 & C \end{cases}$$

转化为矩阵后为：

$$\left| \begin{array}{ccc|c} 1 & 2 & 1 & 7 \\ 2 & -1 & 3 & 7 \\ 3 & 1 & 2 & 18 \end{array} \right|$$

高斯消元求解

对于一个“增广矩阵”要如何求解呢？我们都知道，对于 n 元一次方程，当方程组数至少为 n 时才会有解（可能无解，可能有多组解）。

高斯消元法：

(1) 加减消元，比如如果要消去第一项，那么完全可以让两个 n 元一次等式的第一项系数互为相反数，那么，当这两项相加得到的新的方程就只有 $n-1$ 项了。

(2) 一直消下去，如果有解，那么一定有一个方程只有一个未知数，那么就可以得到答案，再回带求出其他解。

高斯消元过程

0.求解方程组

有这样一个三元一次方程组：

$$\begin{cases} 2x + y + z = 1 & \textcircled{1} \\ 6x + 2y + z = -1 & \textcircled{2} \\ -2x + 2y + z = 7 & \textcircled{3} \end{cases}$$

1.消去x

① $\times (-3)$ + ②得到

$$0x - y - 2z = -4$$

① + ③得到

$$0x + 3y + 2z = 8$$

高斯消元过程

从而得到

$$\begin{cases} 2x + y + z = 1 & \text{①} \\ 0x - y - 2z = -4 & \text{②} \\ 0x + 3y + 2z = 8 & \text{③} \end{cases}$$

2.消去y

② $\times$ 3 + ③得到

$$0x + 0y - 4z = -4$$

进而得到

$$\begin{cases} 2x + y + z = 1 & \text{①} \\ 0x - y - 2z = -4 & \text{②} \\ 0x + 0y - 4z = -4 & \text{③} \end{cases}$$

至此，我们已经求解出来了

$$z = 1$$

高斯消元过程

3.回代求解y

将 $z = 1$ 代入②, 求得

$$y = 2$$

进而得到

$$\begin{cases} 2x + y + z = 1 & \text{①} \\ y = 2 & \text{②} \\ z = 1 & \text{③} \end{cases}$$

最终得到

$$\begin{cases} x = -1 & \text{①} \\ y = 2 & \text{②} \\ z = 1 & \text{③} \end{cases}$$

至此, 整个方程组就求解完毕了。

矩阵模拟

三、再解算法

对于方程组，其系数是具体存在矩阵(数组)里的，下面在给出实际在矩阵中的表示（很熟悉就可以跳过不看啦~）

0.求解方程组

$$\begin{bmatrix} x & y & z & val \\ 2 & 1 & 1 & 1 \\ 6 & 2 & 1 & -1 \\ -2 & 2 & 1 & 7 \end{bmatrix}$$

1.消去x

$$\begin{bmatrix} x & y & z & val \\ 2 & 1 & 1 & 1 \\ 0 & -1 & -2 & -4 \\ 0 & 3 & 2 & 8 \end{bmatrix}$$

2.消去y

$$\begin{bmatrix} x & y & z & val \\ 2 & 1 & 1 & 1 \\ 0 & -1 & -2 & -4 \\ 0 & 0 & -4 & -4 \end{bmatrix}$$

矩阵模拟

3.回代求解y

回代的时候，记录各个变量的结果将保存在另外一个数组当中，故保存矩阵的数组值不会发生改变，该矩阵主要进行消元过程。

$$\begin{bmatrix} x & y & z & val \\ 2 & 1 & 1 & 1 \\ 0 & -1 & -2 & -4 \\ 0 & 0 & -4 & -4 \end{bmatrix}$$

四、再再解算法

说了这么多，其实有一些情况我们还没有说到。

通过上述的消元方法，其实我们比较希望得到的是一个上三角阵(省去了最后的val)

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & -1 & -2 \\ 0 & 0 & -4 \end{bmatrix}$$

下面问题来了：

解的情况

我们都知道，方程组不一定有解，那么当我们使用高斯消元的时候，怎么才能判断是否有解呢？

(1) 如果是一个完美的上三角，那么方程肯定是有唯一解集的

(2) 如果在消元过程中，存在有一行，各项系数都为0，但是常数不为零，那么很明显就是无解

(3) 如果有消元完成之后，有几项系数都为0，并且常数项也为0，那么这几项都可以随意取值，所以方程解不唯一。

(4) 并不是说 n 元一次方程有 n 个方程就一定有唯一解或者无解，也有可能有多组解

异或方程

高斯消元不但可以求解常规方程，还可以求解异或方程。

$$(a[1][1]*x[1]) \wedge (a[1][2]*x[2]) \wedge \dots \wedge (a[1][n]*x[n]) = y[1]$$

$$(a[2][1]*x[1]) \wedge (a[2][2]*x[2]) \wedge \dots \wedge (a[2][n]*x[n]) = y[2]$$

.....

$$(a[i][1]*x[1]) \wedge (a[i][2]*x[2]) \wedge \dots \wedge (a[i][n]*x[n]) = y[i]$$

.....

$$(a[m][1]*x[1]) \wedge (a[m][2]*x[2]) \wedge \dots \wedge (a[m][n]*x[n]) = y[m]$$

异或方程

1. 对于 $k=0..N-1$ ，找到一个 $M[i][k]$ 不为0的行 i ，把它与第 k 行交换。

2. 用第 k 行去异或所有 $M[i][j]$ 不为0的行 i ，消去它们的第 k 个系数，这样就将原矩阵化成了上三角矩阵；

3. 最后一行只有一个未知数，这个未知数就已经求出来了，用它跟上面所有含有这个未知数的方程异或，就消去了所有的这个未知数。

此时倒数第二行也只有一个未知数，它就被求出来了，用这样的方法可以自下而上求出所有未知数。

用矩阵表示大概就是这个样子

线性空间

在一个平面内，给定若干个向量 $a_1, a_2 \cdots a_n$ ，若向量 b 能由 $a_1, a_2 \cdots a_n$ 经过向量加法和标量乘法运算得到，则称向量 b 能被向量 $a_1, a_2 \cdots a_n$ 标出，这样 $a_1, a_2 \cdots a_n$ 能表出的所有向量构成一个线性空间。

任意选出线性空间内的若干个向量，如果其中存在一个向量能被其他向量标出，则称这些向量线性相关，否则称为线性无关。

线性无关的极大子集被称为线性空间的基。简单来说基就是线性空间的极大线性无关子集。一个线性空间内的所有基包含的向量个数都相等，这个数被称为维数。

比如平面直角坐标系中的所有向量构成一个二维线性空间，他的一个基就是单位向量集合 $(0, 1), (1, 0)$ 。

矩阵的秩

关于矩阵的很多性质在大学线性代数中有详细解释，矩阵有一个很重要的东西叫矩阵的秩。秩这个东西的解释很复杂，只需要他的计算方法就可以了，就是把矩阵转换成上(下)三角形式，不为0的行数就叫矩阵的秩。直观理解就是台阶数。下图该矩阵的秩就是3.

The matrix B is shown with its elements arranged in a 5x5 grid. The first three rows are non-zero, and the last two rows are all zeros. The non-zero rows are highlighted by blue ovals around the leading elements: 2 in the first row, 0 in the second row, and 0 in the third row. Red lines connect the leading elements of the first three rows to the first three columns, forming a staircase pattern. The matrix is:

$$B = \begin{pmatrix} 2 & -1 & 0 & 3 & -2 \\ 0 & 3 & 1 & -2 & 5 \\ 0 & 0 & 0 & 4 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

装备购买

对于一个 n 行 m 列的矩阵，我们可以把它的每一行看做一个长度为 m 的向量，称为“行向量”，矩阵的 n 个行向量能够表出的所有向量构成一个线性空间。

脸哥最近在玩一个游戏，游戏里面有 n 件装备，每件装备有 m 个属性，用向量 $z[i]=(a_1, a_2 \cdots a_m)$ 表示，每件装备需要花费 c_i 。现在脸哥想用尽量少的钱买尽量多的装备。

如果一件装备的属性能够用购买的其他装备组合出(用手上已经购买的装备组合出这个效果，可以乘以一个常数)，那么这件装备就没有必要买了。

脸哥想要在买下最多数量的装备的前提下花最少的钱，问如何操作？

装备购买

该题如果明白了矩阵的秩，那么实际上就是求矩阵的秩，或者说求矩阵的非零行的个数。

关键怎么求最少要花多少钱呢？

我们想一下，当我们消到第 i 元的时候，此时下面有很多个前 $(i-1)$ 列都为0，第 i 列不为0的行，那么应该选择哪一个呢？采取贪心策略是正确的，就是用最小的一个价格消去其他的第 i 列的值，相当于选择一个价格最小的装备取尝试组成其他装备。

所以该题就解决了，虽然证明我们还不会。第一步，高斯消元，求出非零行的个数。在消元过程中，每次选择价格最便宜的装备当做基准去消元。

装备购买

该题如果明白了矩阵的秩，那么实际上就是求矩阵的秩，或者说求矩阵的非零行的个数。

关键怎么求最少要花多少钱呢？

我们想一下，当我们消到第 i 元的时候，此时下面有很多个前 $(i-1)$ 列都为0，第 i 列不为0的行，那么应该选择哪一个呢？采取贪心策略是正确的，就是用最小的一个价格消去其他的第 i 列的值，相当于选择一个价格最小的装备取尝试组成其他装备。

所以该题就解决了，虽然证明我们还不会。第一步，高斯消元，求出非零行的个数。在消元过程中，每次选择价格最便宜的装备当做基准去消元。

装备购买

输入：

3 3

1 2 3(每行表示属性)

3 4 5

2 3 4

1 1 2(每件装备价格)

输出：

2 2

1	2	3	1
0	-2	-4	1
0	-1	-2	2

1	2	3	1
0	-2	-4	1
0	0	0	2

最终矩阵的秩只有2，并且使用了第一行和第二行
(当前花费最小的装备)去消元，所以花费为1+1.



练习题

洛谷	3389	高斯消元法
洛谷	4035	球形空间产生器
BZOJ	1830	开关问题
POJ	2811	熄灯问题
洛谷	2962	Light
洛谷	3164	和谐矩阵
洛谷	4111	小Z的房间
洛谷	3265	装备购买
洛谷	4208	最小生成树计数
HDU	3949	XOR



容斥原理

容斥原理主要是用来求解有交并集的集合的一种办法。

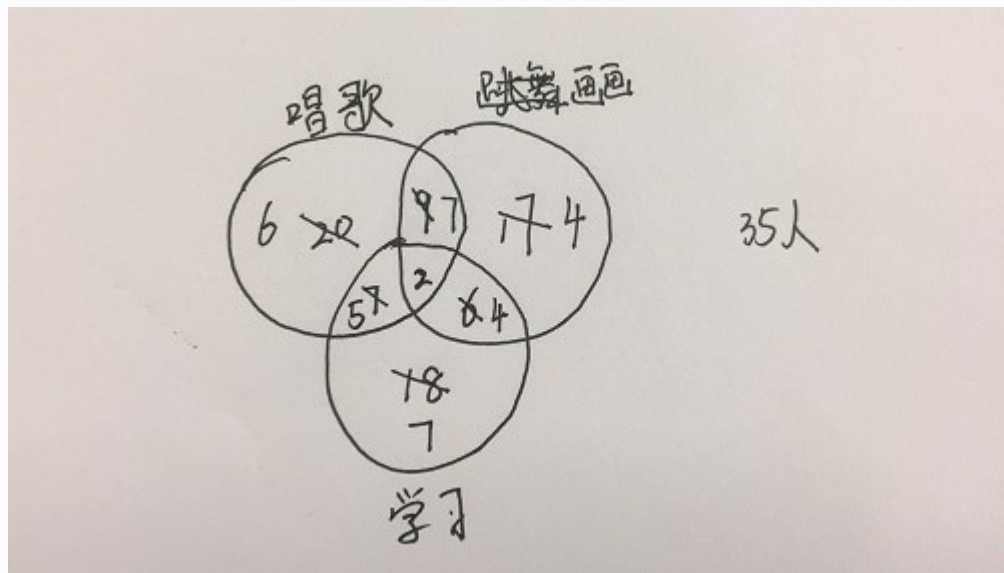
举一个最简单的例子：有20人喜欢唱歌，有17人喜欢画画，有6人既喜欢唱歌又喜欢画画，问一共有多少人。

计算方法很简单： $20+17-6=31$

那么如果再复杂以前呢？有20人喜欢唱歌，有17人喜欢画画，有18人喜欢学习，有9人既喜欢唱歌又喜欢画画，有6人即喜欢画画又喜欢学习，有7人即喜欢学习又喜欢唱歌，有2人三者都喜欢，问一共有多少人？

是不是很晕，虽然我们画一个图一下就知道了，但是如果兴趣种类很多呢？我们该如何计算呢？

容斥原理



计算方法也很简单： $20+17+18-9-7-5+2=35$ 人

实际上容斥原理的计算规则很简单，奇加偶减，如果是奇数个集合的重合，那么就加上，如果是偶数个集合的重合，就减去。比如两个集合的重合就减去，一，三个集合的重合就加上。

容斥原理

给出 m 个数 $a[1], a[2] \cdots a[m]$, 求 $1 \cdots n$ 中有多少个数不是 $a[1], a[2] \cdots a[m]$ 的倍数。

输入:

10 2

2 3

输出:

3

$N \leq 10^9, m \leq 20, a[i] \leq 10^9$

MSOJ 1332 : 计数

容斥原理

这其实就是一道容斥原理的模板题，我们需要求多少个数不是给定的这些数的倍数，那么有多少个数是这些数的倍数呢？如果直接枚举判断肯定超时，那么我们可以每次用给出的数的倍数去筛掉这些数 ($a[i], 2*a[i] \cdots k*a[i]$)，但是最大的问题在于重复，比如一个数很有可能是多个数的倍数，怎么才能既不遗漏又不重复筛选呢？

实际上这就是容斥原理的板子题了，先用 $a[1]$ 去筛选出 $n/a[1]$ 个数，再用 $a[2]$ 去筛选，可以筛选出 $n/a[2]$ 个数，那么重复的怎么算呢？ $n/(a[1]*a[2])$ ，并不是，而是 $\text{lcm}(a[1], a[2])$ ，那么当用第3个数筛选的时候呢？不要忘记还要筛选出 $\text{lcm}(a[2], a[3])$ 和 $\text{lcm}(a[1], a[3])$ ，怎么实现呢？很简单用dfs枚举+剪枝。

容斥原理

```
long long getf(long long x,long long y)
{
    return x*y/gcd(x,y);
}
void dfs(int step,int k,long long lcm)
{
    if(lcm>n)//最小公倍数大于n, 不在考虑了
        return ;
    if(step==m+1)
        //最多选到了m个数, 但是其中可能只选了k个数, 我们实际上是求k个数的交集
        {
            if(k&1)//补集就是奇减偶加
                ans-=n/lcm;
            else
                ans+=n/lcm;
            return ;
        }
    dfs(step+1,k,lcm);//当前第step个数不选, 那此时我选择的数的个数仍然是k
    dfs(step+1,k+1,getf(lcm,a[step]));
    //选择当前第step个数, 那么选择了k+1个数, gcd为前面的lcm和该数的lcm
}
```

多重集的组合数

设 $S = \{n_1 * a_1, n_2 * a_2, \dots, n_k * a_k\}$ 是由 n_1 个 a_1 , n_2 个 a_2 , $\dots$, n_k 个 a_k 组成的多重集。设 $n = \sum_{i=1}^k n_i$, 则对于任意 $r \leq n$, 从 S 中取出 r 个元素组成的多重集方案数为

$$C_{k-r-1}^{k-1} - \sum_{i=1}^k C_{k+r-n_i-2}^{k-1} + \sum_{1 \leq i < j \leq k} C_{k+r-n_i-n_j-3}^{k-1} - \dots + (-1)^k C_{k+r-\sum_{i=1}^k n_i-(k+1)}^{k-1}$$

证明：

不考虑 n_i 的限制，从 S 中任选 r 个元素，相当于从多重集 $S = \{\infty * a_1, \infty * a_2, \dots, \infty * a_k\}$ 取出 r 个元素，则由组合数的知识可得方案数为 C_{k+r-1}^{k-1} 。

设 S_i 为包含至少 $n_i + 1$ 个元素 a_i 的多重集，那么我们先从 S 中选 $n_i + 1$ 个 a_i ，然后任选剩下 $r - n_i - 1$ 个元素，其方案数为 $C_{k+r-n_i-2}^{k-1}$ 。

那么先从 S 中先选 $n_i + 1$ 个 a_i ， $n_j + 1$ 个 a_j ，再任选剩下 $r - n_i - n_j - 2$ 个元素，就可以得到 $S_i \cap S_j$ 的方案数为 $C_{k+r-n_i-n_j-3}^{k-1}$ 。

依次类推，用容斥原理可得至少有一种元素超过限制的方案数为：

$$\left| \bigcup_{i=1}^n S_i \right| = \sum_{i=1}^k C_{k+r-n_i-2}^{k-1} + \sum_{1 \leq i < j \leq k} C_{k+r-n_i-n_j-3}^{k-1} - \dots + (-1)^k C_{k+r-\sum_{i=1}^k n_i-(k+1)}^{k-1}$$

那么合法的方案数就是

$$C_{k-r-1}^{k-1} - \left| \bigcup_{i=1}^n S_i \right|$$

选花

有 N 个盒子，第 i 个盒子里面有 A_i 枝花，同一个盒子内的花颜色相同，不同盒子内的花颜色不同。现在需要选择 M 枝花组成一束花，问有多少种方案、若两束花每种花的颜色都相同，则视为相同方案。输出答案对 10^9+7 取模， $N \leq 20$ ， $M \leq 10^{14}$ ， $A_i \leq 10^{12}$ 。

输入：

3 5

1 3 2

输出：

3

方案为 $(1, 2, 3)$ ， $(1, 3, 1)$ ， $(3, 2)$

取石子游戏

先来一道小学奥数题，有一堆石子，总 n 颗，现在有甲乙两个玩游戏，每次可以取 $1—m$ 颗，但是不可以不取，问是否有什么策略可以保证先手必胜或者后手必胜。

现在来看这道题很简单，如果 n 是 $(m+1)$ 的倍数，那么都采取最优策略，后手必胜。如果 n 不是 $(m+1)$ 的倍数。那么先手必胜，只需要第一次取数让 n 变成 $(m+1)$ 的倍数就可以了。接下来无论对方取多少，你只需要保证你们两个这一轮取的数之和为 $m+1$ 就可以了，那么这样最后当你取完之后就没有石子了。

那么问题如果更加复杂呢？

博弈论

给定 n 堆物品，第 i 堆物品有 A_i 个，两名玩家轮流行动，每次可以任选一堆，取走任意多个物品，可把一堆取光，但不能不取，取走最后一件物品者获胜。两人都采取最优策略，问先手能否必胜。

输入：

5

5 8 6 7 3

输出：

5

4 6 8 3 9

博弈论

博弈论定理： $A_1 \oplus A_2 \oplus \dots \oplus A_n = 0$ 先手必败

证明：当所有物品都被取光是一个必败的局面（对手取走最后一件物品，已经取得胜利），此时每堆石子的总异或为0。

对于任意一个局面，如果 $A_1 \oplus A_2 \oplus \dots \oplus A_n = x \neq 0$ ，如果 x 的最高位为 k ，那么至少存在一堆石子 A_i 的第 k 位为1，并且 $A_i \oplus x < A_i$ 。那我们就从 A_i 中取走若干石子，使其变为 $A_i \oplus x = 0$ ，得到各堆石子异或和=0的局面。

如果 $A_1 \oplus A_2 \oplus \dots \oplus A_n = 0$ ，那无论如何取石子得到的局面下各堆石子异或起来都不等于0。反证法，如果取数之后异或和=0，两个等式1同时合并，除了 A_i 和 A_i' 之外其他都被抵消了，并且两个异或=0，不成立。

博弈论

所以是不是只需要 $a_i' = a_i \oplus x$ 就可以了？注意我们这里是取石子，所以一定要保证 $a_i' < a_i$ ，那么异或一定可以保证 $<$ 吗，肯定不是，还需要满足一定的条件。

设 x 的最高位是第 k 位，如果 a_i 的第 k 位为0，那么异或上去，肯定第 k 位会变成1，无论后面怎么变，都会变大，不满足题意，所以只有需要第 k 位为1的数，异或上去第 k 位为0，一定会变小。

所以简单博弈论就出来了，每次从 $a_i \oplus x < a_i$ 中取 $x - a_i \oplus x$ 个物品数量就可以保证异或之和为0。

博弈论

所以以后玩这游戏就很简单了。先对所有数进行异或，如果异或结果为0，那么先手必败(如果双方都采取最优策略)，如果异或不为0，则先手必胜。

当异或值不为0的时候，比如异或值= x ，并不是说就去取 x 个石子出来就可以了。首先，万一都没有 x 个呢？比如1000 100 10 1，异或= 1111 ，根本就无法取。也不是说异或= 3 ，那么随便找一堆超过3个的取3个就可以了。而是

$$a_1 \oplus a_2 \oplus \dots \oplus a_i \oplus \dots \oplus a_n = x$$

$$a_1 \oplus a_2 \oplus \dots \oplus a_i' \oplus \dots \oplus a_n = 0, \text{ 那么}$$

$$a_i \oplus (a_i') = x。$$

所以这里我们只需要把1000变成111就可以了，异或的结果就为0了

博弈论

给定 n 堆物品，第 i 堆物品有 A_i 个，两名玩家轮流行动，每次可以任选一堆，取走 $1—m$ 个物品，但不能不取，取走最后一件物品者获胜。两人都采取最优策略，问先手能否必胜。

那么这种有限制的取数方法应该如何解决呢？实际上博弈论并不是想象的那么简单。我们前面讲的至只是公平组合游戏的一个特例。

什么是公平组合游戏？

解释起来很复杂，反正就是类似于取石子这种游戏。公平组合游戏都可以通过构造SG函数来完成。

博弈论

我们还是先来看最开始的问题，每次取石子不限制数量，但是必须取。

如果只有一堆石子，那么不用说，先手必胜

那么如果有两堆相同数量的石子呢？先手必败，因为对方可以一直模仿你的动作，那么你一定会输。

所以如果在一局游戏中出现两个相同局面，那么就可以把这两个相同局面消掉构成最简局面。

如果此时存在两个子局面A胜，B负，那么总局面 $S=A+B$ 呢？S必胜，因为局面A必胜，那么最后取子的一定是你，那么相当于对于局面B，对方是先手，那么先手必输，所以最终还是你赢，反过来也是一样。

博弈论

如果子局面A负，B也负呢？不难推出总局面一定是负，推导方法和前面一样。

那么如果子局面A胜，子局面B胜呢？答案是不确定！为什么了呢？

有人会问：为什么不是负呢？局面A先手胜，那么下一局B对方先手，局面是先手必胜，不就是对方胜啊！

你是不是sa！你明知道下一局是先手必胜，那你在A局面你取最后一次干嘛呢？

举一个最简单的例子，一堆石子10， $[1, x]$ ， $[x, 10]$ 都是他的子局面，都是胜，那么先手胜

两堆石子，11枚，分别分成(5, 6)和(6, 5)先手必败

博弈论

看上去很复杂，实际上我们结果和异或有很大关系

A, B局面相同，S负，即 $A=B$, $A \oplus B=0$;

A胜B负，S胜，即 $A \neq 0$, $B=0$, $A \oplus B \neq 0$;

A负B负，S负，即 $A=0$, $B=0$, $A \oplus B=0$

A胜B胜，如果 $A \neq B$, 那么 $A \oplus B \neq 0$

所以总的来看，如果我们要知道一个局面的正负，我们只需要把他的子局面的胜负异或加起来就可以了。

这里我们引入SG函数， $SG(A)$ 表示A局面的二进制数。

博弈论

所以当我们不限制取的数量时，每一个单独的一堆都是先手必胜，我们只需要通过最后是否相同就可以判断，所以只需要直接异或上去就可以了，那么如果我们限制了数量呢？当只有一堆的时候的胜负如何确定呢？

这不就是最开始那道题吗？一堆石子，每次取 $1 \sim m$ 个，问是否可以先手必胜。所以我们只需要对每堆石子 $\% (m+1)$ 就可以然后再把每堆石子异或就可以了。最终的结果和上一题的情况一样

套路：构造f函数

- 用 $\$(x)$ 表示局面 x 下一步所有可能出现的局面的集合

如： $\$(2) = \{(0), \$(1) = (0)\}$

解题关键:如何构造f函数?

- 若 $S=0$, 则使 S 到 T , 一定有 $T \neq 0$
 - 设 $S=(a_1, a_2, \dots, a_n)$, 设先手取了 a_1
 - $S=f(a_1)+\#(a_2, \dots, a_n)=0 \Rightarrow f(a_1)=\#(a_2, \dots, a_n)$
 - 设先手使 $\#(a_1)$ 变为 $\#(b_1, \dots, b_m)$,
 $\#(b_1, \dots, b_m)$ 属于 $\$(a_1)$
 - 则此时局面 $T=\#(b_1, \dots, b_m)+\#(a_2, \dots, a_n)$
即: $T=\#(b_1, \dots, b_m)+f(a_1)$
 - 若要 $T \neq 0$, 则 $\#(b_1, \dots, b_m) \neq f(a_1)$
 - 因此 $f(a_1)$ 的值不属于 $\$(a_1)$

解题关键:如何构造f函数?

- 若 $S \neq 0$, 则先手必能使 S 到 T , 且 $T=0$
 - 设 $S=(a_1, a_2, \dots, a_n), p=s=f(a_1)+f(a_2)+\dots+f(a_n)$
 - 同样 $p \neq 0$, $f(a_k)+p < f(a_k)$, 设 $f(a_1)+p=x$
 - $p=f(a_1)+\#(a_2, \dots, a_n) \Rightarrow \#(a_2, \dots, a_n)=p+f(a_1)=x$
 - 设先手使 $\#(a_1)$ 变为 $\#(b_1, \dots, b_m)$
 $\#(b_1, \dots, b_m)$ 属于 $\$(a_1)$
 - 则此时局面 $T= \#(b_1, \dots, b_m)+\#(a_2, \dots, a_n)$
 - 即 $T=\#(b_1, \dots, b_m)+x$

解题关键:如何构造f函数?

- 若要使 $T=0$, 则找到 $\#(b_1, \dots, b_m)$ 使,
 $\#(b_1, \dots, b_m)=x$
- 若 x 属于 $\$(a_1)$ 则一定能找到
- 又 $x < f(a_1)$, 所以 x 属于集合 $(0, 1 \dots, f(a_1)-1)$
- 若 $\$(a_1)$ 包含这个集合, 则 x 一定属于 $\$(a_1)$

解题关键:如何构造f函数?

所以:

- $f(a_1)$ 不属于 $S(a_1)$
- $S(a_1)$ 包含 $\{0, 1, \dots, f(a_1) - 1\}$

所以 $f(a_1)$ 就是 $S(a_1)$ 关于 N 的补集中的最小值

我们可以用一下函数来模拟

```
for(int i=0;i<=max;i++)  
    if(!book[i])    f[a1]=i;
```



博弈论

有一堆石子 n 个，每次可以取走 $(1, 3, 4)$ 个，问如何判断是否可以先手必胜？



博弈论

我们这里先定义一个 $\text{mex}\{S\}$: 表示集合 S 中没有出现的最小非负整数, $\text{mex}\{0, 2, 3\}=1$, $\text{mex}\{2, 3, 4\}=0$

我们让 $\text{SG}=\text{mex}\{\}$

$N=0$ $\text{SG}[0]=0$

$N=1$ 可以取1个 剩余0个 $\text{mex}\{\text{SG}[0]\}=0$ $\text{SG}[1]=1$;

$N=2$ 可以取1个 剩余1个 $\text{mex}\{\text{SG}[1]\}=0$;

$N=3$ 可以取(1, 3)个, 剩余(0, 2)个, $\text{mex}\{\text{SG}[0], \text{SG}[2]\}$
 $=\text{mex}\{0, 0\}=1$

$N=4$ 可以取(1, 3, 4), 剩余(0, 1, 3), $\text{mex}\{0, 1, 1\}=2$

$N=5$ 可以取(1, 3, 4), 剩余(1, 2, 4), $\text{mex}\{0, 1, 2\}=3$

...



取石子游戏

有3堆石子，每堆有 a_i 颗，每次可以取 $f[i]$ 颗(f 为斐波那契数列)，问是否可以先手必胜。



取石子游戏

```
void get_f()
{
    for(int i=1;i<=16;i++)
    {
        memset(S,0,sizeof(S));
        for(int j=1;j<=1000;j++)
        {
            if(i-a[j]<0) break;
            S[f[i-a[j]]]=1;
        }
        for(int j=0;;j++)
        {
            if(!S[j])
            {
                f[i]=j;break;
            }
        }
    }
}
```

```
int main()
{
    a[0]=1;a[1]=1;
    for(int i=2;i<=16;i++) a[i]=a[i-1]+a[i-2];
    get_f();
    while(1)
    {
        scanf("%d%d%d",&m,&n,&p);
        if(!m&&!n&&!p) return 0;
        if(f[m]^f[n]^f[p]) printf("Fibo\n");
        else printf("Nacci\n");
    }
}
```

总结

遇到取石子这种博弈论的问题：

(1) 如果每次不限制取多少颗，那么根据前面推导出的SG函数，可以知道 $SG[X]=X$. 那么有m堆，只需要把m堆的SG值异或就可以了，实际上就是直接异或

(2) 如果每次可以取1—m颗，推导就会发现 $SG[X]=X\%(m+1)$

(3) 如果每次可以取的数不规则，那么就用前面的推导方法推导一遍就可以了。

详细讲解及推导博客：

<https://www.cnblogs.com/ECJTUACM-873284962/p/6921829.html>

练习题

洛谷	1247	取火柴游戏
洛谷	2197	nim游戏
洛谷	1288	取数游戏2
洛谷	2148	E&D
HDU	1848	Fibonacci again and again
HDU	1847	
HDU	2147	
洛谷	2575	高手过招
洛谷	2594	染色游戏
洛谷	2599	取石子游戏
洛谷	2964	硬币的游戏A
洛谷	4018	取石子



质数相关

给定两整数 L, R , $L, R \leq 2^{31}$, $R - L \leq 10^6$, 求闭区间 $[L, R]$ 中相邻两个质数的差最大是多少, 输出这两个质数



质数相关

由于 n , m 的范围很大, 所以我们没有办法线性筛素数, 但是区间 $[n, m]$ 比较小, 所以我们枚举合数, 怎么枚举呢? 用他的质因子的倍数来枚举。

对于区间 $[n, m]$ 之间的数, 如果他是一个合数, 那么必定有质因子 $\leq \sqrt{m}$, 所以我们只需要筛选出 $1-\sqrt{m}$ 之间的质因数就可以了。在乘以一个整数把每个质因数的倍数在 $[n, m]$ 之间的标记就可以了。最后看下剩下没有标记的数里面差值最大的那个就是。

质数相关

给定整数 N ，试把阶乘 $N!$ 分解质因数，按照算术的基本定理的形式输出分解结果中的 p_i 和 c_i 即可。

$N \leq 10^6$.

质数相关

很容易想到一种暴力方法，分别对1— n 进行质因数分解，在把每个质数的个数累加，但是时间复杂度有点高， $O(N*\sqrt{N})$ ，需要优化。

我们考虑1— n 之间的质因数，直接考虑1— n 中有多少个数是质数 p 的倍数，很明显有 n/p 个，那么 $n!$ 中 p 的次数不是就是 n/p 呢？并不是，因为有的不但是 p 的倍数，还是 p^2 的倍数，但是我们只算了一次，需要加上，加上多少呢？对于 p^2 的倍数，我们只算了一次，但是应该算2次，那么就再加上 $n/(p^2)$ ，那么 p^3 呢？应该算3次，前面算了2次，后面再算一次就可以了，实际上就是

$n!$ 中包含素数 p 的方幂次为：

$$s(n, p) = [n/p] + [n/p^2] + \dots + [n/p^k] + \dots$$

质数相关

很容易想到一种暴力方法，分别对1— n 进行质因数分解，在把每个质数的个数累加，但是时间复杂度有点高， $O(N*\sqrt{N})$ ，需要优化。

我们考虑1— n 之间的质因数，直接考虑1— n 中有多少个数是质数 p 的倍数，很明显有 n/p 个，那么 $n!$ 中 p 的次数不是就是 n/p 呢？并不是，因为有的不但是 p 的倍数，还是 p^2 的倍数，但是我们只算了一次，需要加上，加上多少呢？对于 p^2 的倍数，我们只算了一次，但是应该算2次，那么就再加上 $n/(p^2)$ ，那么 p^3 呢？应该算3次，前面算了2次，后面再算一次就可以了，实际上就是

$n!$ 中包含素数 p 的方幂次为：

$$s(n, p) = [n/p] + [n/p^2] + \dots + [n/p^k] + \dots$$

练习题

P0J 2689 Prime Distance

P0J 2992 阶乘分解

BZ0J 1053 反素数

BZ0J 1257 余数之和

P0J 3090

P0J 3696

BZ0J 1607 轻拍牛头

P0J 2262 哥德巴赫猜想

BZ0J 樱花

BZ0J 1053 Antiprime数

BZ0J 3629 聪明的燕姿

洛谷 1066 2^k 进制数

BZ0J 3398 牡牛和牝牛

BSGS算法

BSGS算法是解决一类类似于求：

$A^x \bmod p = B$ 中关于 x 的最小正整数解的算法

其中 p 是一个质数。他有一个很有意思的名字，称为“大步小步”算法，他的算法思想也体现了“大步”与“小步”。由于算法过于强大，也被称为“拔山盖世”算法，“北上广深”算法等等。

BSGS算法

求解： $A^x \bmod p = B$

其中A, B, p是已知量。其中A和p是互质的。

我们先考虑一下暴力如何做？

很简单，就是从1开始枚举x，但是枚举到什么时候结束呢？如果找到一个满足条件的解，那么就可以直接结束，但是，如果无解呢？那么我们需要枚举到什么时候才可以判断是真的无解还是还没有枚举到答案呢？

BSGS算法

我们可以猜测当 x 一直增加的时候， $A^x \% p$ 的余数会循环，实际上真的是这样。这个是可以数学方法证明的

根据费马小定律可以知道：

当 A 和 p 互质时 $A^{(p-1)} \% p = 1$

那么我们就可以知道 $A^p \% p = A^{(p-1)} * A \% p = A^1 \% p$

也就是说最多从1枚举到 $p-1$ 就可以了，也可以视为 $[0, p-2]$ ，因为继续向下面枚举一定是重复的，但是这只是一个循环节，并不一定是最小的循环节，实际上根据欧拉定理 $A^{(\phi(p))} \% p = 1$ ，所以这种算法的时间复杂度是 $O(p)$ 的，如果 p 特别大，那么效率就会很低。

BSGS算法

我们假设 $x=km-r$ ，其中 $r<m$ ，相当于 m 是除数， r 是余数（有一点区别，原理是一样的）

那么原等式 $A^x \% p=B$ 就可以写出：

$A^{(km-r)} \% p=B$ ，等式两边同时乘以 A^r ，得到：

$A^{(km)} \% p=B*A^r \% p$ ，因为 $r<m$ 的，所以 r 的范围是 $(0—m-1)$ ，对于任意的 m ，一定可以找到一个满足条件的 $x=km-r$ ，我们可以先求出所有 $B*A^r \% p$ 的答案，只需要计算 m 次就可以了。

接下来，我们只需要计算左边 $A^{(km)}$ 的值可能有哪些情况就可以了，因为 $km-r$ 的范围是 $(0—p-1)$ ，在 m 固定的情况下， k 的可能情况大概有 p/m 种，对于每一种情况，我们都去右边查询下这种结果是否存在。只要有一种情况满足条件，那么就说明有解。

时间复杂度

计算右边的所有结果的时间复杂度为 $O(m)$ ，也就是与我们选择的除数有关，计算左边的所有结果的时间复杂度为 $O(p/m)$ ，为了让总体时间复杂度最低，那么 m 取 $\sqrt{p}$ 。

对于每一次左边计算的结果，我们需要去右边的所有可能性中查询答案，这里我们可以用map，也可以用hash，map的时间复杂度为 $O(\log n)$ ，hash时间复杂度为 $O(1)$ ，计算的幂次方的时候可以每一次都快速幂去计算，实际上因为 m 是固定的，所以我们可以先余出来出来所有 $A^k \pmod m$ ， A^m 就可以了，所以总体最快时间复杂度为 $O(\sqrt{p})$ 。

代码

这里讲用map存数，最好自己用hash存储，map的效率特别低，虽然他的本质是平衡树，但是他的效率远低于手写平衡树，时间复杂度差不多是两个log，甚至更慢，比赛的时候慎用，10w的数据都容易被卡。

```
#include<map>
map<int int> mp;//定义了一个map数组mp
map.clear();//多组数据，使用前先清空
//求 $a^x \pmod p = b$ ,  $x = km - r$ ，等式变换之后就是
// $a^{(km)} \pmod p = a^r * b \pmod p$ 
mp[b%p]=0;//0就代表这个值
for(int i=1;i<m;i++)
{
    t=t*a%p;
    mp[t]=i;//i表示r， $a^r * b$ 的答案为mp[t]
    //m.insert(make_pair(t,i))
    //感觉下面的写法才是正确的
    //数组赋值形式以最后一次为准，后面形式以第一次为准
}
```

代码

接下来枚举 k ，范围是 $1 \sim m$ ，因为需要求 a^{km} 的值， m 是固定的，所以我们可以先求出 a^m 的值，后面直接用。

```
//a^(km)%p=a^r*b%p
//a^(km)=(a^m)^k
long long mi=quick(a,m,p);//(a^m)^k
t=1;
for(int i=1;i<=m;i++)//相当于枚举k
{
    t=t*mi%p;
    if(mp[t])//如果存在
    {
        ans=(i*m-mp[t])%p;//x=km-r
        printf("%lld\n", (ans+p)%p);//可能是负数
        return ;
    }
}
```



练习题

洛谷	4884	多少个1
洛谷	2485	计算器
洛谷	P3846	可爱的质数
洛谷	4195	拓展BSGS



中国剩余定理

孙子算经：今有物不知其数，三三数之余二，五五数之余三，七七数之余二，问物几何。

也就是说，一堆物品，3个3个的数，还剩下2个，5个5个的数，还剩下3个，7个7个的数，还剩下2个，问这堆数最少有多少个？

同时，古人也给出了计算口诀：三人同行七十稀，五树梅花廿一枝，七十团圆月正半，除百零五便得知。

也就是说：3个3个的数的余数 $\times 70 + 5$ 个5个的数的余数 $\times 21 + 7$ 个7个的数的余数 $\times 15$ ，最终得到的答案对105取余的答案就是正解。

关键是，这个倍数是怎么算出来的呢？

中国剩余定理

一、首先，%105很容易理解， $105=3*5*7$ ，如果 x 是正确答案，那么 $x+105*k$ 一定也是正确答案。

二、接下来是如何算倍数。

我们假设 x 是正解，那么 x 一定可以写成 $2*x_1+3*x_2+2*x_3$ 之和，并且：

(1) x_1 满足 $\%3=1, \%5=0, \%7=0$

(2) x_2 满足 $\%3=0, \%5=1, \%7=0$;

(3) x_3 满足 $\%3=0, \%5=0, \%7=1$;

只要满足上式之后， x 一定也满足最前面的取模的答案。

我们只需要分别算出满足条件的 x_1, x_2, x_3 就可以求解了。

中国剩余定理

三、我们先计算第一个等式

x_1 满足 $\%3=1, \%5=0, \%7=0$;

那么很明显, x_1 是 35 的倍数, $x_1=35y$, 并且 $x_1\%3=1$

即 $35*y\%3=1$, 根据分配率 $(33y+2y)\%3=1$

所以 $2y\%3=1$, 同时乘以 2, 在减去 3 得到

最终 $y\%3=2$, 很明显 $y=2$, 那么 $x_1=70$

我们可以用同样的方法算出 x_2, x_3 :

$x_2=21y; 21*y\%5=1; y\%5=1; y=1 \quad x_2=21$

$x_3=15y; 15*y\%7=1; y\%7=1; y=1 \quad x_3=15$

根据上式, 我们就很轻松的推导出来了倍数的求解方法, 我们就可以继续推导 n 阶中国剩余定理的求解方法了。



中国剩余定理

一道水题：POJ 1006



中国剩余定理

对于一般的等式，我们只需要：

对于一个数 x_i ，我们先求出 $x_1 * x_{i-1} * \dots * x_{i+1} * x_n$ 的答案为 a (要求互质，就等于相乘)

设自然数 $m_1, m_2, \dots, m_r$ 两两互素，并记 $N = m_1 * m_2 * \dots * m_r$ ，则同余方程组：

$$\begin{cases} x \equiv b_1 \pmod{m_1} \\ x \equiv b_2 \pmod{m_2} \\ \dots \\ x \equiv b_r \pmod{m_r} \end{cases}$$

在模 N 同余的意义下有唯一解。

证明：考虑方程组 ($1 \leq i \leq r$)：

$$\begin{cases} x \equiv 0 \pmod{m_1} \\ \dots \\ x \equiv 0 \pmod{m_{i-1}} \\ x \equiv 1 \pmod{m_i} \\ x \equiv 0 \pmod{m_{i+1}} \\ \dots \\ x \equiv 0 \pmod{m_r} \end{cases}$$

通过变量替换，令 $x = (N/m_i) * y$ ，方

只需要再计算出 $a * x_i \% lcm = 1$ 的最小正整数解就可以了。很明显可以通过拓展欧几里得来求解。最后把得到的 x_i 的值 * 除数加起来初一最小公倍数就可以了。

注意限制条件为： a_i 都互质，否则不可以用。

猜数字

现有两组数字，每组 k 个，第一组中的数字分别为： $a_1, a_2, \dots, a_k$ 表示，第二组中的数字分别用 $b_1, b_2, \dots, b_k$ 表示。其中第二组中的数字是两两互素的。求最小的非负整数 n ，满足对于任意的 i ， $n - a_i$ 能被 b_i 整除。

输入：

3

1 2 3

2 3 5

输出：

23

猜数字

现很容易看出来这是一道中国剩余定理的模板题。 $(n-a[i])\%b[i]=0$ 那么 $n\%b[i]=a[i]$. 求解最小非负整数 n 满足条件

需要注意的地方：

(1) $a[i]$ 并不是一定是整数，但是 $a[i]$ 表示余数，为了消除影响，需要把 $a[i]$ 表示最小正整数。

$(a[i]\%b[i]+b[i])\%b[i]$.

(2) 同样，得到的 x 的值也需要保证最小非负整数

(3) 因为只保证了数据范围为 10^{18} ，但是并没有保证中间计算结果也在这个范围内，所以不能直接乘，需要快速乘（类似于快速幂）。

拓展中剩

但是理想是丰满的，现实是骨感的。出题人才懒得出保证除数都互质的数据呢？一般都是随机给出除数，这种情况明线不可以直接使用中国剩余定理，那需要怎么办呢？

我们可以考虑方程合并，也就是通过一组解求出其他组解，实际上，在操作过程中和中国剩余定理没有太大的关系。

拓展中剩

$x=k_1*a_1+b_1$; $x=k_2*a_2+b_2$; 联立得:

$$k_1*a_2+b_1=k_2*a_2+b_2;$$

$$a_1*k_1-a_2*k_2=b_2-b_1;$$

未知量只有 k_1 和 k_2 , 是不是很熟悉, 替换一下:

$$a_1*x-a_2*y=b_2-b_1; \text{求最小正整数解}$$

只要 b_2-b_1 是 $\gcd(a_1, a_2)$ 的倍数, 就有正整数解

我们用拓欧可以得出其中一组解 (k_1, k_2) ;

$$\text{设 } d=b_2-b_1 \quad g=\gcd(a_1, a_2);$$

$$a_1*x_1-a_2*y_2=d;$$

$$a_1*k_1+a_2*y_2=g;$$

$$\text{所以 } x_1=k_1*d/g \quad y_1=-k_2*d/g;$$

其中最小正整数解为 $(x_1 \% (a_2/g) + a_2/g) \% (a_2/g)$;

合并方程

已知解 x , 我们可以把当前这两个方程合并:
合并之后 $a1$ 和 $b1$ 分别为:

$$a1 = x * b1 + a1;$$

$$b1 = (b1 * b2) / g;$$

洛谷 P4777: 模板

洛谷 P2183: 礼物

洛谷 4774: 屠龙勇士(拓展再拓展)

Lucas定理

Lucas定理是用来求 $C(n, m) \% p$ 的值，其中， n 和 m 是非负整数， p 是素数，一般用于 m, n 很大， p 很小，或者 $n, m > p$ ，这种情况，这样以前的方法就会出现问题的（如果某个位置取余后为0，后面的答案都是错误的。）

Lucas定理：

$$\text{lucas}(n, m, p) = C(n \% p, m \% p) * \text{lucas}(n/p, m/p, p);$$

在计算组合数的时候，因为是对 p 取余，所以就不需要担心上面 $=0$ 的问题，直接用费马小定律求解，如果想更快的话，可以用阶乘逆元。递归求解

$$\text{lucas}(n/p, m/p, p)$$



Lucas 定理

洛谷 P3807 : lucas 定理

HDU 3037 : saving beans

HDU 5226 : Tom and matrix

洛谷 P2480 古代猪文





拓展 Lucas 定理

拓展 Lucas 定理当 p 为非质数的时候的组合数取余的问题，他的原理是唯一分解定理+中国剩余定理+Lucas 定理来求解。

